Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Kushaal Shyam Potta	Register No.	3122235001071
Due Date	09.01.2026		

Experiment 2: Binary Classification using Naïve Bayes and K-Nearest Neighbors

1 Aim and Objective

To build and evaluate Naïve Bayes (Gaussian, Multinomial, and Bernoulli) and K-Nearest Neighbors (KNN) classifiers for the binary classification task of spam detection. The experiment involves data preprocessing, exploratory data analysis, hyperparameter tuning using Grid and Randomized search, and performance comparison using metrics such as Accuracy, Precision, Recall, F1-Score, and Specificity.

2 Dataset Description

The experiment utilizes the **Spambase** dataset, which consists of features extracted from emails to classify them as spam or non-spam.

- **Total Instances:** 4601 (Original), 4210 (After removing duplicates)
- **Total Features:** 57 numerical features (word frequencies, character frequencies, run length attributes) + 1 target class
- **Target Variable:** 'class' (1 for Spam, 0 for Non-Spam)

Dataset Reference: Kaggle – Spambase Dataset

3 Preprocessing Steps

- **Data Loading:** The dataset was loaded using the Pandas library.
- **Duplicate Removal:** 391 duplicate rows were identified and removed to ensure data quality, reducing the dataset size to 4210 instances.
- **Feature Scaling:** The `StandardScaler` was applied to normalize the features for Gaussian Naïve Bayes and KNN models. Multinomial Naïve Bayes utilized the original non-negative features.
- **Train-Test Split:** The data was split into training and testing sets to evaluate model performance on unseen data.

4 Implementation Details

4.1 Naïve Bayes

Three variants were implemented:

- *GaussianNB:* Applied to scaled data for continuous features.
- *MultinomialNB:* Applied to raw count/frequency data.
- *BernoulliNB:* Applied to binary/boolean feature representations.

4.2 K-Nearest Neighbors (KNN)

- A baseline KNN model was trained.
- **Hyperparameter Tuning:** `GridSearchCV` and `RandomizedSearchCV` were used to find the optimal k value, distance metric, and weight function.
- **Search Algorithms:** Evaluated using *KDTree* and *BallTree* structures for efficiency.

5 Visualizations

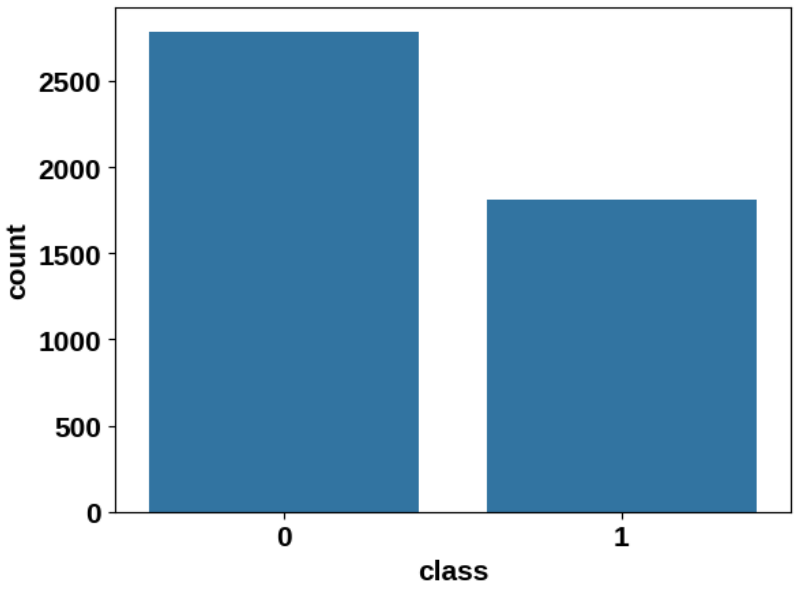

Figure 1: Class Distribution of Spam and Non-Spam Emails

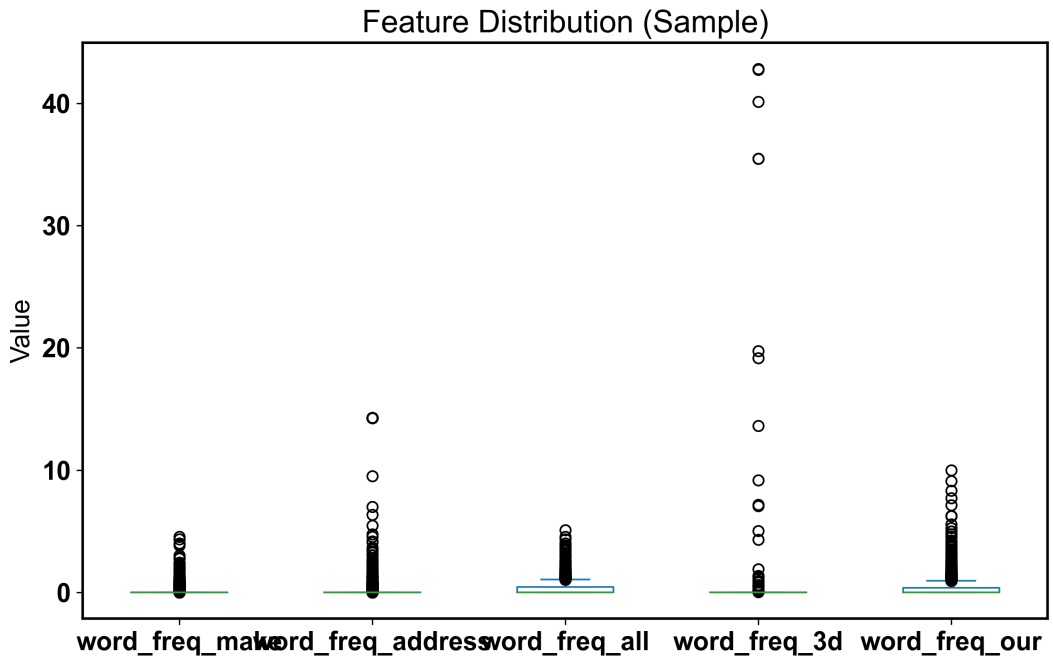

Figure 2: Feature Distribution Plot (Box plot)

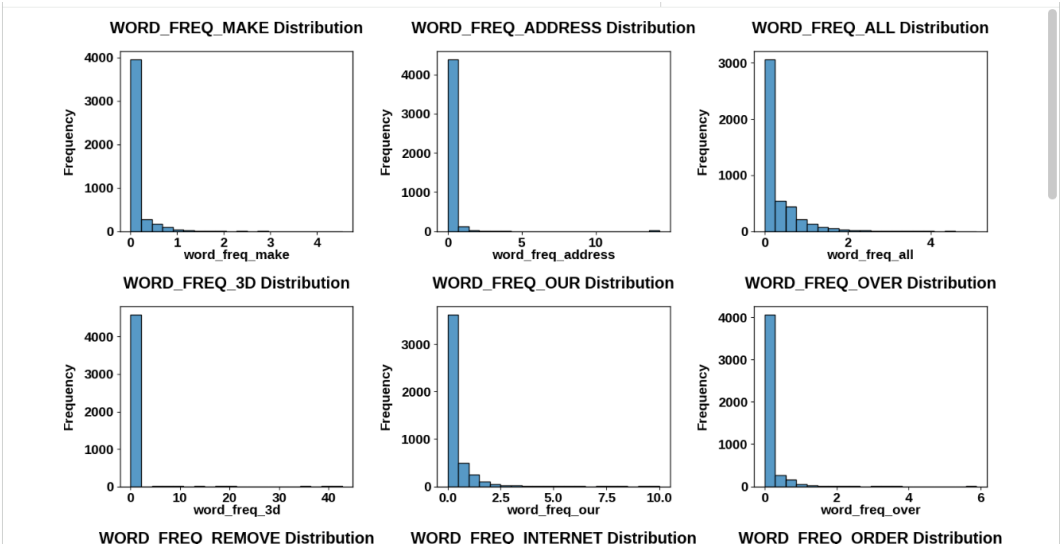

Figure 3: Feature Distribution Plot (Histogram)

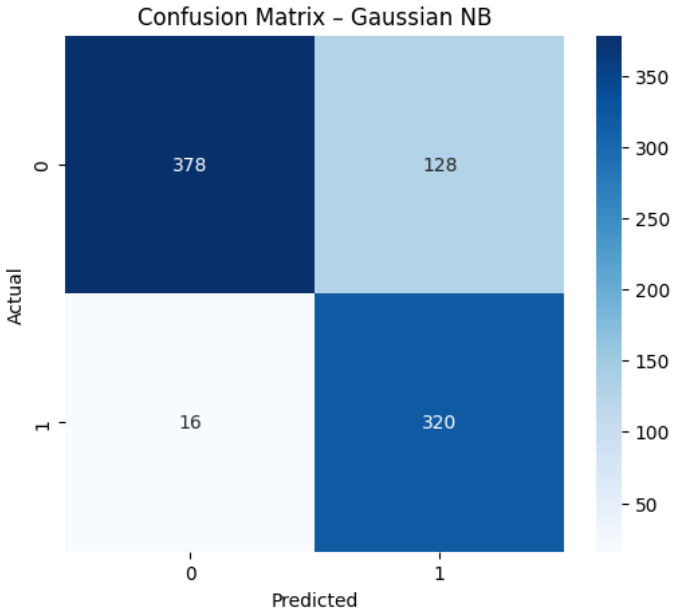

Figure 4: Confusion Matrix for Gaussian Naïve Bayes

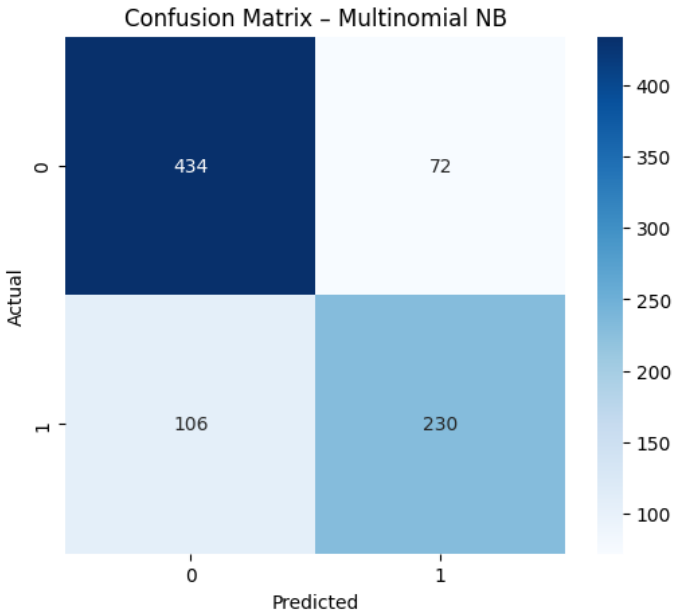

Figure 5: Confusion Matrix for Multinomial Naïve Bayes

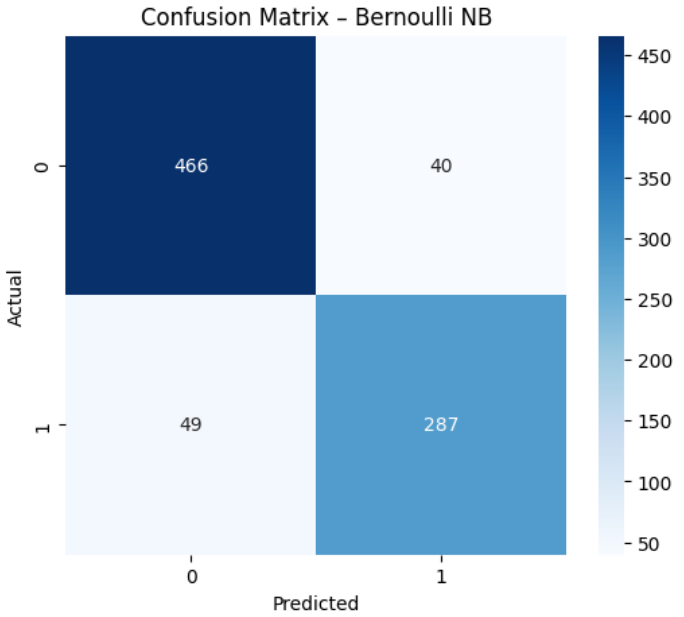

Figure 6: Confusion Matrix for Bernoulli Naïve Bayes

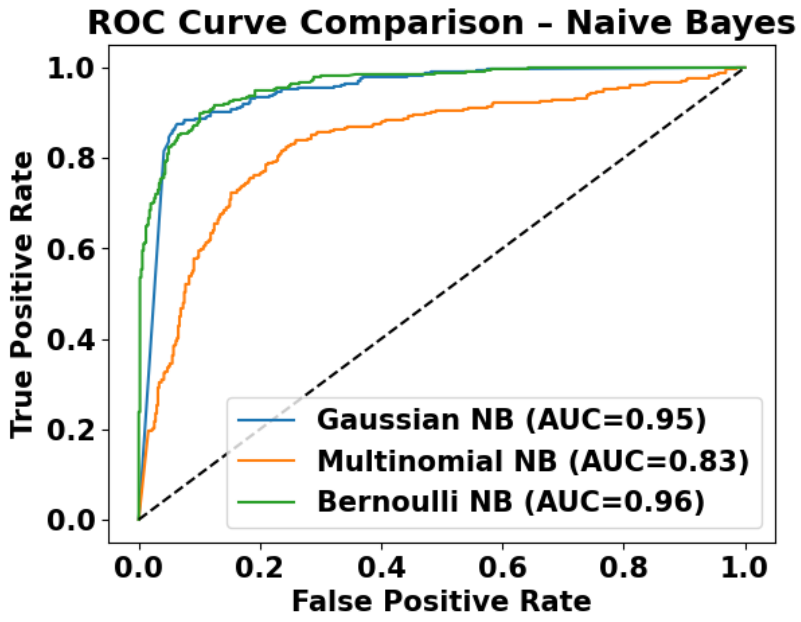

Figure 7: ROC Curve for Naïve Bayes Models

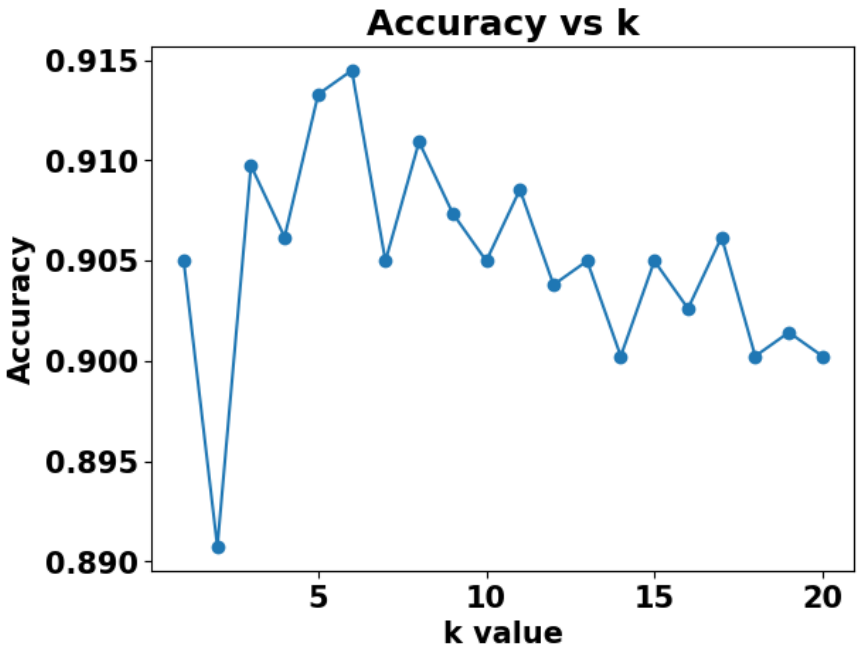

Figure 8: Accuracy vs. k Plot for KNN

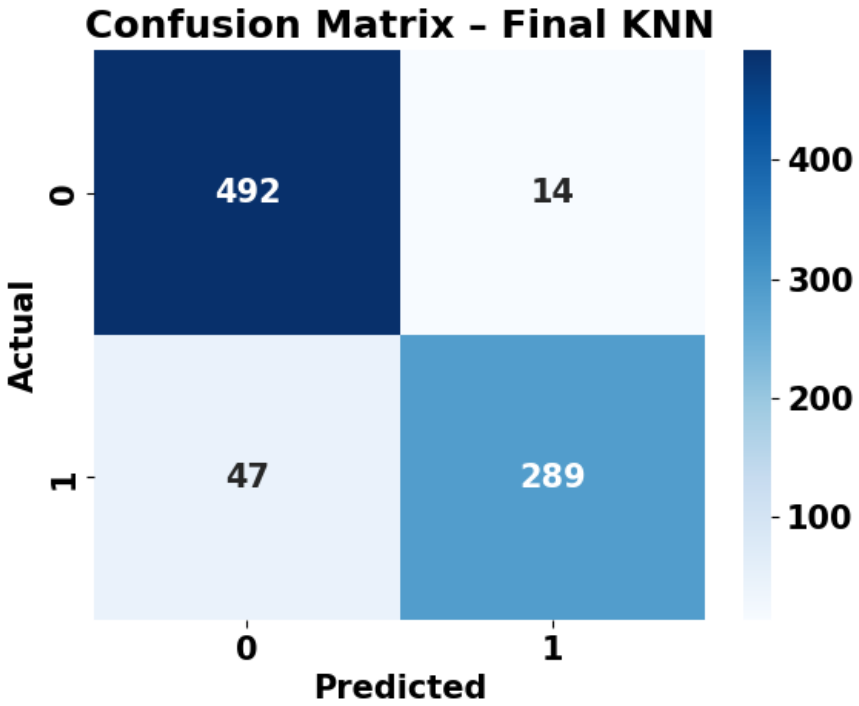

Figure 9: Confusion Matrix for Final KNN

6 Performance Tables

Table 1: Naïve Bayes Performance Metrics

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8290	0.7886	0.8943
Precision	0.7143	0.7616	0.8777
Recall	0.9524	0.6845	0.8542
F1 Score	0.8163	0.7210	0.8658
Specificity	0.7470	0.8577	0.9209
Training Time (s)	0.0031	0.0033	0.0043

Table 2: KNN Hyperparameter Tuning

Search Method	Best Parameters	Best CV Accuracy
Grid Search	k=7, weights='distance', metric='manhattan'	0.9181
Randomized Search	k=6, weights='distance', metric='manhattan'	0.9127

Table 3: KNN Performance using KDTree (Final Model)

Metric	Value
Optimal k	7
Accuracy	0.9276
Precision	0.9538
Recall	0.8601
F1 Score	0.9045
Training Time (s)	0.0013
Prediction Time (s)	0.0179

Table 4: KNN Performance using BallTree (Final Model)

Metric	Value
Optimal k	7
Accuracy	0.9276
Precision	0.9538
Recall	0.8601
F1 Score	0.9045
Training Time (s)	0.0013
Prediction Time (s)	0.0179

Table 5: Comparison of Neighbor Search Algorithms

Criterion	KDTree	BallTree
Accuracy	0.9276	0.9276
Training Time (s)	0.0013	0.0013
Prediction Time (s)	0.0179	0.0179
Memory Usage	Low / Medium	Medium / High

7 Overfitting and Underfitting Analysis

- **Underfitting:** Observed in Multinomial Naïve Bayes, where the assumption of feature independence and integer counts may not fully capture the complexity of the continuous features in the dataset, leading to lower recall (0.6845).
- **Overfitting:** In KNN, very small values of k (e.g., $k = 1$) often lead to overfitting as the model becomes too sensitive to noise in the training data. The grid search identified $k = 7$ as optimal, balancing bias and variance.
- **Optimal Balance:** The Bernoulli NB and Optimized KNN models showed robust performance on the test set (Accuracy $> 89\%$), indicating good generalization without significant overfitting.

8 Bias–Variance Analysis

- **Gaussian NB:** High Recall (0.9524) but lower Precision (0.7143) indicates the model is biased towards the positive class (Spam), capturing most spam emails but with more false positives.
- **Multinomial NB:** Lower Recall (0.6845) suggests high bias against the spam class structure in this specific feature set.
- **KNN:** The distance-weighted KNN with Manhattan distance ($k = 7$) achieved the highest Specificity (0.9723) and F1 Score (0.9045), demonstrating low bias and low variance after tuning.

9 Observations and Conclusion

The experiment demonstrated that for the Spambase dataset:

- **KNN (Tuned)** outperformed all Naïve Bayes variants with an accuracy of **92.76%** and an F1-score of **0.9045**.
- Among Naïve Bayes classifiers, **Bernoulli NB** performed best (Accuracy: 89.43%), suggesting that the presence/absence of words is a strong predictor for spam.
- Hyperparameter tuning significantly improved KNN performance, with `GridSearchCV` identifying Manhattan distance and distance-based weights as optimal.
- Gaussian NB showed the highest Recall, making it suitable if the cost of missing a spam email is high, though at the cost of precision.

10 References

1. Scikit-learn – Naïve Bayes Documentation
2. Scikit-learn – KNN Documentation
3. Scikit-learn – Hyperparameter Optimization
4. Kaggle – Spambase Dataset